Session-Level Inference Control for LLM Agents on Analytical Databases

Sanjay Mishra
Independent Researcher
Raleigh, NC, USA
sanmish4@icloud.com

ABSTRACT

Agents built on large language models (LLMs) now write SQL against analytical databases for analysts. Some data, such as customer balances, may be released only in aggregate, and analytical databases enforce this one query at a time, for example with a minimum group size. But answers combine: two averages whose groups differ by one customer give that customer's balance. We present a *session audit* in a gateway in front of the agent's SQL tool. It records which individuals each answer covered and refuses a query whose answer, combined with earlier ones, would determine one individual's value. Other answers are returned exactly. The guarantee is exact recovery, per identity, on a static snapshot. On the finance sessions the linear audit refuses 3.3% of benign queries and 26.8% of benign sessions, against 0.6% and 7.0% for a per-query minimum group size, and it stops the scripted differencing attacks that the minimum group size does not. A portfolio of 100 targets and at most 100 queries, with no language model, recovered every target from that minimum group size and from nested pairs, and none from the linear audit. Snowflake and BigQuery were not measured here: this repository has no account for them. Cached attackers `gpt-4o-mini` (50 targets) and `gpt-4o` (25) disclosed no target under the linear audit. `claude-sonnet-5-5` refused the attacker role and sent no SQL.

PVLDB Reference Format:
Sanjay Mishra. Session-Level Inference Control for LLM Agents on Analytical Databases. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/sanmish4ds/least-privilege-llm-db-agents.

1 INTRODUCTION

A retailer shares customer data with a marketing partner, or an HR team publishes pay statistics. In both cases the data may be used in aggregate but never per person. Analytical databases support this directly: Snowflake's aggregation policies, BigQuery's aggregation thresholds and AWS Clean Rooms' minimum aggregation refuse any answer whose group contains fewer than k distinct individuals [5, 20, 45]. Increasingly, the queries come from an agent built on a large language model (LLM), which writes SQL, sends it to a database tool over the Model Context Protocol (MCP) and summarizes the answers. Such an agent writes many queries per task, reads every refusal and tries again, and can be steered by instructions hidden in text it reads [23, 38].

These controls judge each query on its own, but disclosure can come from a combination. Two averages whose groups differ by one customer reveal that customer's value, although each average covers many people. Statistical databases called this the tracker problem [13] and built audits for it [8, 27], but those audits assumed a static query interface and never reached products. Vendors know the risk: Snowflake's documentation notes that a determined user with enough queries could work around its aggregation requirements [45], and Google's that its analysis rules are not guaranteed to block every query designed to extract raw data [21]. Their remedies are differential privacy with a query budget [4, 20, 46], which makes every answer approximate, and pre-approved query templates [21], which rule out the free-form SQL an agent writes. Wang et al. name the same failure V8, lack of compositional leakage control: a policy that approves each query can still leak through a sequence of aggregates [52]. Role-aware text-to-SQL benchmarks judge one query against a fixed role [17, 55], and monitors for multi-turn attacks watch the dialogue [54]. None of this tracks what a session's exact SQL answers have already released.

Fig. 1 shows the pattern in plain form. The agent asks for an average balance over a market segment in one nation, then the same average over the customers with keys below 1,400. Both are ordinary analyst questions, both cover a dozen customers, and a minimum group size of three admits both. Subtracting the two totals gives the balance of the one customer who is in the first group and not the second. The lower panel needs no arithmetic at all: a count that is 0 or 1 says whether one customer's balance is above a threshold, and halving the interval each time pins the balance to a cent in about 21 questions.

Unlike a multi-turn jailbreak such as Crescendo [40], whose build-up a monitor can see in the dialogue [54], every step here is innocuous text; the disclosure exists only in the arithmetic relation between answers. A defense therefore has to keep the answers, and specifically the sets of individuals behind them.

This paper places a gateway between the agent and the database. The gateway enforces the aggregate-only rule and keeps a *session audit*: for every answer that depends on a protected column it records which individuals the answer covered, and it refuses a query whose answer, alone or combined with earlier ones, would rest on fewer than k individuals or determine one individual's value. In Fig. 1a the audit refuses the second question, because the two groups differ by one customer; in Fig. 1b it refuses the first, because the count depends on one customer's balance. Answers that single

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

(a) Differencing

A: What is the average balance of BUILDING customers in nation 7?
DB: 4,512.30, over 12 customers.
A: And for the same customers with key below 1,400?
DB: 4,301.10, over 11 customers.
A: $12 \times 4{,}512.30 - 11 \times 4{,}301.10 = 6{,}835.50$: customer 1,412's balance.

(b) Threshold bisection

A: How many customers with key 570 have a balance above 5,000?
DB: 1.
A: …above 7,500? …above 8,750?
DB: 1. 0.
A: (18 more halvings) The balance is known to the cent.

Figure 1: Two ways a session leaks one customer's balance through questions that are each allowed (illustrative values; A: agent, DB: database answer). No single query returns the balance, and no query text names the goal.

no one out are returned exactly. That query history matters is old: linear auditing of statistical databases decided it for sum queries over explicit index sets [8]. The claim of this paper is that this auditing, carried to the free-form SQL an agent writes, bounds exact linear disclosure from aggregate-only data. On the finance sessions that costs more benign refusals than a per-query minimum group size (Section 5.3).

The guarantee is narrower than the experiments: admitted sums and averages never determine any linear function of fewer than k individuals' values, but filters on a protected column are audited without proof, and a refusal can depend on protected values. *Strict mode* refuses those filters, so every decision depends on non-protected data only. Table 3 lists each SQL feature with what it releases, the result that covers it and what remains.

On the finance sessions a per-query minimum group size does not stop differencing, and the linear audit does (Section 5.3). A 100-target portfolio with no language model recovers every target from minimum group size and from nested pairs, and none from the linear audit (Section 5.2). Cached LLM attackers that read every refusal did not reach an exact value under the linear audit (Section 5.4). Snowflake and BigQuery were not run. This is evidence for the attacks in the repository, not a bound beyond Table 3.

This paper makes the following contributions:

(1) **A measured gap in per-query minimum group size.** On the finance sessions and on a 100-target portfolio, a minimum group size does not stop differencing. The linear audit does (Sections 5.2 and 5.3). Native Snowflake and BigQuery controls are not in this repository.
(2) **A session audit for free-form SQL.** The linear test is the classic audit expert [8]; what is new is the input and the small-support test. A rewrite turns free-form SQL, including joins, CTEs, filters on protected columns and conditional aggregates, into weighted sets of the individuals behind each answer. A combination test, pruned by a leverage bound, refuses answers that would determine any linear function of fewer than k individuals' values. We prove both tests exact, show that the decision is NP-complete when k is part of the input, and give a guarantee for each SQL feature (Sections 4.3 and 4.4).
(3) **Evidence against scripted, adaptive and rule-aware attackers.** LLM attackers that adapt to refusals, a programmatic attacker aimed at the audit's known gaps, and a comparison with differential privacy (Sections 5.3–5.5).
(4) **A deployable MCP gateway and its cost.** The audit runs on a read-only connection or as a proxy; we measure it on human-written BIRD gold queries and end to end on PostgreSQL at TPC-H scale factor 1 (Sections 5.6 and 5.6).

Scope. The gateway does not decide which columns a user or task may see; the database's own controls do that. It does not detect prompt injection. It answers exactly or refuses. It assumes the gateway and the database account are not themselves compromised. The ledger is per authenticated identity and per static snapshot.

2 RELATED WORK

2.1 Aggregate-only controls in analytical databases

Database access control decides which rows and columns a user may see: role-based access control [42], fine-grained control through authorization views [39], and in current products row and column policies, masking and, for agents, the end user's identity passed into the database [37]. These controls cannot express "aggregates only": a column is either visible or not, and masking leaks through filters on the masked column, as vendor documentation itself warns [33]. Analytical databases add aggregate-only rules. Snowflake's aggregation policies [45] and BigQuery's aggregation threshold [20] refuse groups with fewer than k distinct entities, and AWS Clean Rooms enforces a minimum aggregation per output row [5]. Each rule applies to one query's result, and none records what earlier queries covered. For protection across queries the same products offer differential privacy with a budget [4, 20, 46]. Some clean rooms restrict partners to pre-approved query templates, which removes differencing but also the free-form SQL an agent writes. Purpose-based access control [3, 7] and anomaly detection over SQL activity [25] likewise judge queries one at a time.

2.2 Inference control and query auditing

Answering only large-group aggregates is not enough: a tracker combines allowed queries to isolate an individual [13], and limiting how much query sets overlap delays such a compromise without preventing it [15]. The audit expert decides by linear algebra whether a set of answered sum queries determines an individual value [8]; exact auditing is computationally hard in general, even when the values are Boolean [28], and the early controls are compared in [1]. Enough noise-free aggregate answers allow reconstruction of a database [14], and refusals themselves can leak unless the audit decision is simulatable from public information [27]. Differential privacy replaces exact answers with noisy ones [16] and has been engineered for SQL [24]; k-anonymity protects released tables rather than query streams [50]. Diffix, a SQL proxy that suppresses low counts and adds noise fixed per query condition [18], was still defeated by differencing and reconstruction attacks [9, 19], because its filter judges each query on its own. Auditing was later

carried to SQL. SQL auditors find which past queries accessed specified data [2], decide online whether a conjunctive query is suspicious [34], or make audit trails efficient for SQL with grouping and subqueries [26]. Online auditors exist for max and min queries and their utility for sums has been analyzed [35, 36], and interval-based inference is intractable to audit for bounded integers and needs mathematical programming for bounded reals [30]. These SQL auditors decide whether a query accessed or depended on sensitive tuples; the linear auditors decide determinability for sum queries over explicit index sets. Our audit sits between them: it takes free-form SQL with joins, filters and conditional aggregates, derives the weighted entity sets behind each answer, and decides online, for exact answers, how few entities the session's answers can single out. The weight vector is the provenance polynomial of the aggregate, specialized to a sum [6, 22]; we use it only to decide refusal. QuerySnout and QueryCheetah search for attribute-inference queries against systems that answer aggregates [11, 48]. The portfolio in the appendix is a much smaller, hand-specified set of those patterns, not their search. None of the classical audits was evaluated against agents that read refusals and adapt.

2.3 Security of LLM data agents

Wang et al. list V8, lack of compositional leakage control: a policy that approves each query never tracks what the answers reveal together [52]. This paper is a defense for that channel, not a claim that the channel was unnamed. SafeNLIDB trains a model to refuse multi-turn aggregation inference [31]. SecureSQL finds that a chain-of-thought checker does not stop multi-turn leakage from LLMs used as database interfaces [47]. Both checks are learned or prompted; this audit does not depend on the model. FactGate (first draft: SessionBound) accounts for released facts and states that inference from predicates is not bounded [53]. DPSQL+ pairs differential privacy with a minimum frequency [32]. PIMENTO does the same pairing for text [41]. We do not run them. BIRD supplies the human-written queries below [29].

2.4 Multi-turn attacks and agent authorization

Crescendo [40] opens with an innocent question and asks the model to build on its own replies until it produces what it would have refused if asked directly; monitors that accumulate evidence across a decomposed request detect such attacks better than per-prompt filters [54]. Against prompt injection, CaMeL attaches capabilities to values derived from the trusted request [12], Progent enforces per-task privilege policies on tool calls [43], FIDES propagates confidentiality labels through an agent's planner [10], and authorization architectures scope tool access per task and track authority consumed by earlier calls [44, 49]. In these systems a database answer is one opaque value, and what several answers reveal together about one row is outside the model. The gateway is complementary: it can be the database tool such a system calls.

2.5 Positioning

Table 1 summarizes the comparison: only auditing keeps exact answers and decides from earlier ones, and the session audit is the one that accepts a fragment of free-form SQL and was run against adaptive attackers.

Table 1: Positioning. *Exact*: answers are not perturbed. *Free SQL*: arbitrary queries rather than templates or sum queries (fragment: refuses shapes it cannot audit). *Session*: decisions use earlier answers. *Adaptive*: run against an attacker that reads each answer and refusal and chooses its next query, and measured by whether it recovered an exact value (model: such an attacker is analyzed in a formal model but not run).

Line of work	Exact	Free SQL	Session	Adaptive
Row/column control, masking [33, 37, 39]	yes	yes	–	–
Per-query thresholds [5, 20, 45]	yes	yes	–	–
Low-count proxy [18]	–	yes	–	–
Differential privacy [16, 20, 24]	–	yes	budget	–
Linear auditing [8, 27, 36]	yes	sums	yes	model
SQL auditing [26, 34]	yes	yes	access	–
SafeNLIDB [31]	yes	yes	learned	multi-turn
SecureSQL checker [47]	yes	yes	chat	weak
FactGate [53]	yes	yes	fact budgets	traces
DPSQL+ [32]	no	fragment	accountant	–
Session audit (this work)	yes	fragment	answers	yes

3 PROBLEM AND THREAT MODEL

This section states who the adversary is and which property the gateway protects. It uses three terms. An *entity* is one of the individuals of Section 1: one row of the table that owns a sensitive value, such as one customer. A column is *protected* if its per-entity values may only be aggregated, never returned. The parameter k, set by the deployment, is the smallest number of entities an answer may rest on.

3.1 Threat model

The data owner allows a protected column to be used only in aggregate. The adversary controls the agent's inputs (the question, content the agent retrieves) or the agent itself. It may issue any number of queries in a session, read every answer and refusal, and adapt. It cannot change the gateway or the database account, and it cannot open unlimited sessions under one identity, because the audit state is keyed to an identity the deploying application authenticates.

3.2 Protected property

Session disclosure has two parts. (i) No admitted answer that depends on a protected column rests on fewer than k entities, as a group, as the difference of two nested groups, or through a filter on the protected column. (ii) No set of admitted answers determines exactly any linear function of fewer than k entities' protected values: never one entity's value and, for $k \geq 3$, never a sum, difference or other combination of a few.

Part (ii) is proved for sums (Section 4.4). For filters it holds for the candidate sets, which bounds which entities an answer can depend on but not how answers at different thresholds combine; strict mode removes filters on protected columns altogether. The property assumes the protected values do not change while the gateway keeps its record of earlier answers, the *ledger* (Section 4.1): inserted and deleted entities change the key sets and are caught, but a changed value is not, so a deployment audits a read-only snapshot and renews the ledger with it.

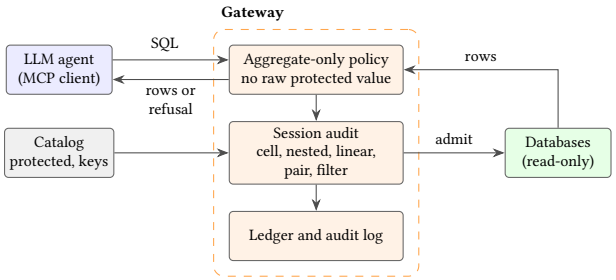


Figure 2: Gateway between LLM agents and databases. The policy check refuses a query that returns a protected column; the session audit refuses a query whose answer would rest on fewer than k entities given earlier answers, using the ledger of entity sets behind admitted answers; only admitted queries reach the read-only connection.

4 METHOD

The gateway applies two checks to every query: the aggregate-only policy, which refuses a query that returns a protected column, and the session audit, which decides whether its answer would single anyone out given earlier answers. This section gives an overview of the gateway, describes each check, states what is guaranteed, and closes with the full per-query procedure and its cost.

4.1 Gateway overview

Fig. 2 shows the gateway. Agents reach databases only through it. It is an MCP server with two tools, `begin_session(purpose)` and `run_sql(sql)`. It executes admitted queries either itself, on a connection whose role has only SELECT and a read-only transaction default, or by forwarding them to a downstream database MCP server, so it can sit in front of an existing vendor server (Section 5.6). A catalog supplies keys, entity tables and the protected columns.

Sessions. A session starts with a purpose, which names the protected columns, k, and the identity the session runs under. The gateway keeps two pieces of state for it: the ledger of entity sets behind admitted answers, per protected column, and the set of entities whose sensitive columns were returned raw, checked against the purpose's entity budget. A refused query changes neither, because nothing was released. Calling `begin_session` again, with the same or another purpose, keeps the ledger, since it is keyed by protected column rather than by purpose; and the ledger is stored per authenticated identity, so a new connection under the same identity does not start from an empty ledger either.

4.2 Aggregate-only policy

A purpose declares which columns are *protected*. Like an aggregation policy in an analytical database, it allows a protected column inside SUM and AVG and refuses any query that returns it. These two are linear in the protected values, which is what the session audit models. A protected column inside any other aggregate is refused wherever it appears, including HAVING: MIN, MAX, MEDIAN, percentiles and list aggregates can return one entity's exact value, and VARIANCE and STDDEV release a second moment, so an average of 100 with a variance of 0 fixes every value in the group however large it is. The gateway classifies each use o of a column c in a query q as raw if o is returned (in the outer select list outside any aggregate, or the argument of a projected MIN or MAX), agg if o is inside a value aggregate, and filter otherwise (WHERE, JOIN, GROUP BY, ORDER BY, or inside COUNT). Columns are resolved to $t.c$ by qualifying q against the schema, so aliases, joins and subqueries are handled; a query that cannot be resolved is refused. A query that uses a protected column in mode raw, or a column outside the purpose, is refused before it runs.

Example. In Fig. 1a the balance is used in mode agg and the segment, nation key and customer key in mode filter, so both queries pass this rule. A query returning one customer's balance would not. The policy alone therefore stops direct reads but not the combination of two allowed averages; that is the job of the session audit.

Table 2: Notation.

Symbol	Meaning
P, $E(p)$	Protected columns; entity table and key of p
$\mathcal{S}(q)$, $C(q)$	Entity sets of q's groups; candidate sets of q's protected filter
$\mathcal{H}$, $\mathcal{H}_\downarrow$	Sets of earlier admitted aggregates and protected filters
k	Minimum number of entities an answer may rest on
w_S, Q	Weights of group S (rows per entity); orthonormal basis of answered weights

4.3 Session audit

The audit asks one question before each query runs: if this answer is released, how few people could it be about, given everything already released? It answers with entity sets. Let $p \in P$ be a protected column of entity table $E(p)$ with key κ. For an admitted query that aggregates p, the gateway rewrites it to select κ and the grouping expressions, dropping aggregation, HAVING, ORDER and LIMIT, and runs the rewrite on the read-only connection. This yields $\mathcal{S}(q) = \{S_1, \ldots, S_m\}$, one set per output group, and for each a weight vector w_S, where $w_S(i)$ is the number of rows entity i contributes to the group. Joins repeat entities: a SUM of balances over `customer JOIN orders` is $\sum_i w_S(i)\, x_i$ with $w_S(i)$ the customer's order count; without joins every weight is 1. That vector is the provenance of the answer as a weighted sum [22]. Five tests follow.

Table 2 collects the symbols used in the rest of this section.

Small-cell test. The first test asks how many entities the smallest group in the answer covers:

$$\iota_1(q) = \min_{S \in \mathcal{S}(q)} |S|. \tag{1}$$

Nested-pair test. Take Fig. 1a. The first answer covers 12 customers and the second covers 11 of the same 12. Neither group is small, but exactly one customer is in the first and not the second, and subtracting the two sums gives that customer's balance. The nested-pair test finds every pair of answers in which one group contains the other and counts the entities in the difference. With $\mathcal{H}$ the groups of earlier admitted aggregates over p,

$$\iota_2(q) = \min_{(A,B) \in N(q)} \left|\{i : w_B(i) > w_A(i)\}\right|, \tag{2}$$

where $N(q)$ is the set of pairs from $\mathcal{S}(q) \cup \mathcal{H}$ with $w_A \le w_B$ entrywise and $w_A \neq w_B$, at least one of them in $\mathcal{S}(q)$. In Fig. 1a this count is 1, so for any $k \ge 2$ the second query is refused.

Linear test. Differences need not be nested: three overlapping groups, none containing another, can still be added and subtracted to leave one entity, and Section 5.4 shows attackers finding such combinations. The linear test therefore treats each answer as one equation in the unknown per-entity values and asks whether the equations admitted so far, together with the new one, can be solved for any single entity's value. A SUM answer is the inner product of w_S with the vector of per-entity values; an AVG is the same once the group's COUNT is known. Entity i's value is determined by the answers iff the unit vector e_i lies in their span [8]. With Q an orthonormal basis of $\{w_S : S \in \mathcal{H}\}$,

$$e_i \in \operatorname{span}(Q) \iff (QQ^\top)_{ii} = \textstyle\sum_j Q_{ij}^2 = 1, \tag{3}$$

tested with tolerance 10^{-9} on normalized weight vectors, and the set of entities q would newly disclose is $D(q) = \{i : (Q'Q'^\top)_{ii} = 1\}$, where Q' extends Q by Gram–Schmidt with w_S for $S \in \mathcal{S}(q)$. The weights are taken only from the bare column: SUM or AVG of an expression of p (a product, a CASE, DISTINCT) is not a weighted sum the audit can model, and is refused. One shape is the exception, because it is a filtered sum in disguise: a conditional aggregate such as `SUM(CASE WHEN` c `THEN` p `ELSE 0 END)` returns the same value as `SUM(`p`) WHERE` c. The gateway audits such a query as several queries: one filtered aggregate per conditional aggregate, plus the remainder with each conditional aggregate replaced by a constant. ORDER, LIMIT, HAVING and DISTINCT are dropped from each part, which can only enlarge its groups' exposure, never shrink it. The parts are checked in turn against the session state, and the query is admitted only if every part is; if one is refused, nothing from the earlier parts is kept.

Combination test. The linear test stops one entity's value, but an attacker can settle for a combination of a few. Suppose customers t and u appear together in one group with three others, R_1, and the agent also asks for $R_1 \cup R_2$ and for R_2 alone. No group is small, no nested pair differs by fewer than three customers, and neither e_t nor e_u lies in the span, yet the first answer minus the second plus the third is $x_t + x_u$. Swapping one customer for another in an otherwise equal group gives $x_t - x_u$ in the same way, and the same three queries with m targets give the sum of their m balances. The combination test refuses a query after which some linear function of fewer than k entities' values would be determined. Such a function exists exactly when some set T of fewer than k entities is essential to the answers: deleting T's entities from every group would lower the rank of the answered weights.

The test finds such sets through *leverage* $L_i = \|Q_i\|^2$, the squared norm of row i of Q; $L_i = 1$ is the linear test's condition, and the leverages sum to the rank r. A combination supported on T needs $\sum_{i \in T} L_i \ge 1$, so T contains an entity with $L_i \ge 1/|T|$, and at most $|T|\, r$ entities qualify. For two entities the condition has a closed form,

$$a\, e_i + b\, e_j \in \operatorname{span}(Q) \iff (1 - L_i)(1 - L_j) = (Q_i \cdot Q_j)^2, \tag{4}$$

because the 2×2 block of $QQ^\top$ on $\{i, j\}$ must have eigenvalue 1; only entities with $L_i \ge \frac{1}{2}$, at most $2r$ of them, need partners. For a combination of up to $s > 2$ entities, the test takes each *anchor* i with $L_i \ge 1/s$, deletes coordinate i from the span, and searches what remains for a combination of up to $s - 1$ entities, ending in Eq. 4. Deleting row i keeps the basis orthonormal after a rank-one correction, $Q_{-i}\big(I + (1/\sqrt{1 - L_i} - 1)\, Q_i^\top Q_i / L_i\big)$, at cost $O(nr)$. Let $D_{<k}(q)$ be the sets of fewer than k entities this search finds once q's sets are added; for $k \le 2$ it is empty, since the linear test already covers one entity.

Filter test. A protected column can leak without being aggregated. In Fig. 1b the query counts rows, not balances, but the count is over one customer, so it is 0 or 1 according to whether that customer's balance passes the threshold. The filter test asks which entities' protected values the answer depends on, and applies the three tests above to those entities. When q uses p in its WHERE clause outside any aggregate, the gateway *relaxes* q: it replaces every comparison on p with TRUE, which yields a query $q^\downarrow$ that matches a superset of q's rows. The candidate sets $C(q)$ are the entity sets of $q^\downarrow$'s output groups, or one singleton per entity if q returns rows rather than aggregates. The gateway applies Eqs. 1–4 to $C(q)$ against a separate history $\mathcal{H}_\downarrow$, giving $\iota_\downarrow(q)$ and $D_\downarrow(q)$. In Fig. 1b the relaxed query matches one customer, so its candidate set holds one entity and the first probe is refused.

Relaxation gives a superset only for comparisons joined by AND and OR, so a protected column under NOT or CASE, in JOIN ON, GROUP BY, ORDER BY or HAVING, or in a subquery, is refused. So is SUM, AVG, MIN or MAX of a non-protected column under such a filter: summing known, distinct order totals over the customers whose balance passes a threshold would decode which of them passed. Unequal weights decode it too (a `COUNT(*)` over a join where the target has three orders and two others one each shows whether the target passed), so every candidate in a group must carry the same weight unless the query is `COUNT(DISTINCT` κ`)`.

The gateway refuses q iff

$$\min(\iota_1(q), \iota_2(q), \iota_\downarrow(q)) < k \text{ or } D(q) \cup D_\downarrow(q) \cup D_{<k}(q) \cup D_{<k\downarrow}(q) \neq \emptyset. \tag{5}$$

A refused query reveals nothing, so its sets enter no history. Eq. 1 alone is the stateless baseline (cell suppression); Eqs. 1–2 form the pairwise audit; Eqs. 1–3 the linear audit; Eq. 5 in full adds the combination test.

Nested queries and fail-closed coverage. The gateway merges every CTE and derived table that only filters, joins or renames into the query that reads it. Each one left that reads a protected column, typically a total per group, is audited as an answer of its own before the query that reads it, and the query is admitted only if every part is. The final answer then depends on protected values only through admitted answers, so it reveals nothing beyond them, at the cost of refusing a total per person even when the final answer only averages it. A protected column that a CTE passes on outside an aggregate counts as returned. The gateway refuses what it cannot attribute: a protected or sensitive column inside a set operation or recursive CTE, the entity table referenced twice within one part, nested queries over protected columns in strict mode (a filter on a CTE's total would make decisions depend on protected values), and any query whose rewrite fails. A rewrite that matches no entity is admitted, since the answer covers no one.

Table 3: Guarantee per SQL feature over a protected column p. All rows assume values fixed while a ledger lives; none covers values in non-protected columns or collusion across identities.

SQL feature	Released	Covered by	What remains
SUM, AVG of p (joins, CASE, nesting)	sums Ax	Props. 4.1, 4.2	intervals if values are bounded
p in a filter or COUNT	counts over thresholds	tests, no proof	across thresholds; strict refuses
Refusals	who matches	strict: Prop. 4.4	may leak if not strict
p returned, or in MIN, MAX, MEDIAN, VARIANCE, STDDEV, other aggregates or an expression	–	refused	–
Set operation, recursive CTE, failed rewrite	–	refused	–

Entity budget. Some purposes need individual records: a support agent helping one customer must see that customer's details. Such a purpose carries an entity budget. The gateway counts the distinct entities whose sensitive columns were returned raw in the session and refuses a query that would return them for more entities than the purpose allows.

4.4 Guarantees

We state what the audit guarantees precisely; Table 3 summarizes it per SQL feature.

Proposition 4.1. *Let $A \in \mathbb{N}^{m\times n}$ have the weight vectors of the m admitted aggregate groups over a protected column as rows, and let each answer be the exact sum Ax of the unknown per-entity values $x \in \mathbb{R}^n$. If no admitted query satisfied $D(q) \neq \emptyset$, then for every entity i and every x there is x' with $Ax' = Ax$ and $x'_i \neq x_i$.*

Proof. Each admission checked Eq. 3 with the query's sets added, so after the last admission $e_i \notin \mathrm{row}(A)$ for every i; refused queries add no rows. Since $\mathrm{row}(A) = \mathrm{null}(A)^{\perp}$, $e_i \notin \mathrm{row}(A)$ implies some $z \in \mathrm{null}(A)$ has $z_i \neq 0$. Put $x' = x+z$. Then $Ax' = Ax$ and $x'_i = x_i + z_i \neq x_i$. The test itself is exact because $(QQ^{\top})_{ii} = \|QQ^{\top}e_i\|^2$ equals 1 iff e_i lies in the span. □

Proposition 4.2. *Under the same conditions, if no admitted query satisfied $D_{<k}(q) \neq \emptyset$, then no nonzero linear function of the values of fewer than k entities is determined by the answers.*

Proof. As in Proposition 4.1, $c^{\top}x$ is determined iff $c \in V = \mathrm{row}(A) = \mathrm{span}(Q)$. (a) If $c \in V$ has unit norm and support T, then $1 = c^{\top}QQ^{\top}c \leq \lambda_{\max}\big((QQ^{\top})_{TT}\big) \leq \sum_{i\in T} L_i$, so some $i \in T$ has $L_i \geq 1/|T|$ and is tried as an anchor. (b) Let $L_i < 1$ and let V_{-i} be V with coordinate i deleted. If $c \in V$ is supported on $T \ni i$, then $c_{-i} \in V_{-i}$ is nonzero (otherwise $e_i \in V$ and $L_i = 1$) and supported on $T \setminus \{i\}$; conversely, a nonzero $d \in V_{-i}$ supported on T' is c_{-i} for some $c \in V$ supported on $T' \cup \{i\}$. So V has a nonzero vector on at most s entities through i iff V_{-i} has one on at most $s-1$. (c) $Q_{-i}^{\top}Q_{-i} = I - Q_i^{\top}Q_i$, whose inverse square root is the correction above, so the recursion works on an orthonormal basis of V_{-i}. Induction on s, with Eq. 3 and Eq. 4 as base cases, shows the search is exact. Refused queries add no rows. □

The search is exponential in k, and no algorithm can avoid that in general.

Theorem 4.3. *Given $A \in \mathbb{N}^{m\times n}$ and k, deciding whether some nonzero vector of* $\mathrm{row}(A)$ *is supported on fewer than k entities is NP-complete.*

Proof. It is in NP: a set T is a certificate, checked by testing whether deleting the columns T lowers the rank of A. For hardness we reduce from deciding whether a rational matrix $W \in \mathbb{Q}^{d\times n}$ has a circuit of at most $s < n$ columns, that is, a nonzero z with $Wz = 0$ and at most s nonzero entries, which is NP-complete [51, Thm. 1]. Compute a rational basis $N_1, \ldots, N_t$ of $\mathrm{null}(W)$ and let $V \subseteq \mathbb{R}^{2n}$ be spanned by the vectors $(N_j, 0)$ and the all-ones vector $\mathbf{1}$. Every vector of V is $(z + \lambda\mathbf{1}, \lambda\mathbf{1})$ with $Wz = 0$; if $\lambda \neq 0$ it has at least $n > s$ nonzero entries, and if $\lambda = 0$ it is $(z, 0)$. So V has a nonzero vector on at most s entries iff W has a circuit of at most s columns. The vectors $(N_j, 0) + \mu\mathbf{1}$ for a large rational μ, together with $\mathbf{1}$, are nonnegative and still span V; scaled by a common denominator they form A, of size polynomial in that of W. Set $k = s + 1$. □

So no audit runs in time polynomial in both n and k unless P = NP. The search above is polynomial for each fixed k: level s tries at most $s\,r$ anchors, so it runs at most $\frac{(k-1)!}{2}\, r^{k-3}$ pair searches (Eq. 4), each $O(nr^2)$, with the exponent on the rank rather than on the number of entities. The reduction uses general nonnegative weights, as joins produce; whether the problem stays hard for plain sets, with every weight 0 or 1, we leave open.

The filter test has no such proof; exact filter auditing is hard even for Boolean values [28]. A count under a threshold is linear in the indicator vector of that threshold, $[x_j > t]_j$, and different thresholds give different unknown vectors. The audit treats all filters on p as one unknown, which refuses every single-entity probe and every nested or linear combination of candidate sets, but an attacker who combines answers across thresholds is not covered by Proposition 4.1. Strict mode closes that gap.

Strict mode. Refusals can leak too: the decision depends on which entities match a query, and if membership depends on protected values, a refusal says something about them [27]. In strict mode the gateway refuses any query that uses a protected column outside an aggregate.

Proposition 4.4. *In strict mode, the sequence of gateway decisions for a session is a function of the queries and of the non-protected columns only.*

Proof. The aggregate-only policy and the strict check depend only on the query text. For an admitted aggregate, the key-only rewrite reads no protected column; the entity sets $\mathcal{S}(q)$ and their weights, and hence Eqs. 1–5, depend only on non-protected data. By induction over the session, $\mathcal{H}$ and Q do too. □

4.5 Per-query procedure

Algorithm 1 puts the policy check and the session audit together in the order the gateway applies them.

4.6 Cost

Parsing and qualification are linear in the query size. Each rewrite costs one extra query whose result size is the number of matching rows. The linear test keeps $Q \in \mathbb{R}^{n\times r}$ over the n entities touched

Algorithm 1 Gateway check for query q under a purpose

resolve every column of q to $t.c$; **if** unresolvable **return** refuse
if q touches a protected or sensitive column inside a set operation or recursive CTE **return** refuse
merge CTEs and derived tables that only filter or rename; split the rest into parts, each CTE over a protected column first and q last; check each part in turn, and keep none if one is refused ▷ Sec. 4.3
if q returns a protected column, uses one in a non-linear aggregate, or uses a column outside the purpose **return** refuse ▷ Sec. 4.2
if strict and q filters on a protected column **return** refuse
split conditional aggregates into parts; check each part in turn, and keep none if one is refused
for $p \in P$ used by q in a filter **do**
    $C \leftarrow$ candidate sets of the relaxed rewrite; **if** it fails **return** refuse
    if $\iota_{\downarrow} < k$ **or** $D_{\downarrow}(q) \cup D_{<k\downarrow}(q) \neq \emptyset$ **then return** refuse ▷ Eq. 5
    end if
end for
for $p \in P$ used by q in mode agg **do**
    $\mathcal{S} \leftarrow$ entity sets of the key-only rewrite; **if** it fails **return** refuse
    if more groups than the limit **return** refuse
    if $\min(\iota_1, \iota_2) < k$ **or** $D(q) \cup D_{<k}(q) \neq \emptyset$ **then return** refuse ▷ Eq. 5
    end if
end for
if sensitive columns returned raw for more entities than the budget **return** refuse
commit $\mathcal{S}, C$ to $\mathcal{H}, \mathcal{H}_{\downarrow}$ and their bases; save the ledger; execute q; log
return rows

so far and rank $r \leq n$; adding a set costs $O(nr)$ with two Gram–Schmidt passes for stability, and Eq. 3 costs $O(nr)$. A query with more groups than a configurable limit (2,000 in our runs) is refused rather than audited. Table 4 measures the check: 0.05 ms with 1,000 entities and rank 10, and 28 ms with 50,000 entities and rank 500, where the dense basis takes 200 MB. Those rows are the committed scaling log. A one-million-entity dense-basis timing is not in the repository, so it is not reported. Weights are integers, so the 10^{-9} test in Eq. 3 can be replaced by rational elimination. On 400 random integer-weight trials the two agreed (0 disagreements); on a constructed pair with entries near 10^9 the floating-point test missed a disclosure the rational test caught (appendix). The rates below were computed with the floating-point test.

5 EVALUATION

The evaluation asks five questions, one per subsection: whether a per-query minimum group size suffices (RQ1), what each test prevents and costs (RQ2), whether the audit holds against adaptive and rule-aware attackers (RQ3), whether alarms or noise would do instead (RQ4), and what it costs in practice (RQ5).

Table 4: Linear-audit check time (median of 5) and basis memory vs. entities n and rank r, for random contiguous key-range sets (Python/NumPy, Apple M-series CPU, single process).

n	r	Check (ms)	Basis (MB)
1,000	10	0.05	0.1
1,000	100	0.09	0.8
1,000	500	0.28	4.0
10,000	10	0.27	0.8
10,000	100	0.83	8.0
10,000	500	5.37	40.0
50,000	10	1.24	4.0
50,000	100	6.07	40.0
50,000	500	28.27	200.0

5.1 Setup

Environment. Python 3.14, sqlglot 30.23, SQLite, PostgreSQL 16, NumPy, and the MCP Python SDK. Model ids are the strings in the cached attacker files; dated snapshots were not stored. `gpt-4o-mini` issued SQL on 50 targets and five defenses, at most 15 queries, temperature 0.7. `gpt-4o` issued SQL on 25 targets; strict mode was not in that file. `claude-sonnet-5-5`, called as an attacker on one target and five defenses, returned a refusal on the first turn of each and sent no SQL. No model judges disclosure: the rank test does.

Data. PostgreSQL at scale. TPC-H at scale factor 1 (150,000 customers) on PostgreSQL 16, with 200 random target customers and three benign workload mixes of 1,600 queries each. *TPC-H sessions.* A seeded TPC-H-shaped SQLite database (1,500 customers, 7,500 orders, synthetic values) has four purposes and a session generator: 1600 benign analyst sessions (400 per purpose) with realistic noise (30% contain one query outside the purpose's columns; finance sessions include drill-downs and overlapping histogram bands) and 500 attack sessions in five families (drift, small cell, two-query differencing, three-query differencing, enumeration; the figures call differencing "tracker"). Every session has 6–16 queries regardless of label, so length carries no label information. *BIRD sessions.* The human-written gold queries of BIRD's [29] `financial` (106 queries) and `debit_card_specializing` (64) databases are arranged into 200 random sessions of 6–16 queries per database.

Adaptive attackers. Each attacker is an LLM told it is red-teaming a privacy gateway over synthetic data and given one customer key; it must find that customer's exact balance in at most 15 queries, seeing rows or refusal reasons after each.

Baselines. The audit is compared in five modes: the small-cell test alone (*stateless*); the small-cell and nested-pair tests (*pairwise*); the small-cell, nested-pair, linear and filter tests (*linear*); the linear audit with no filter on protected columns (*strict*); and the linear audit with the combination test (*final gateway*). For comparison with intent-level monitoring we also compute session scores: drift, the sensitivity-weighted number of distinct columns used outside the purpose; cell, $1/\min(\iota_1, \iota_2)$ over the session; and breadth, the number of distinct entities whose protected or sensitive columns were returned raw, divided by the purpose's entity budget.

Metrics. An attack session is *prevented* if the gateway refuses its disclosing query or an earlier one. A benign query is a *false refusal*

Table 5: Portfolio on 1,500 synthetic customers, 100 targets, budget 100, no language model. *Exact*: the target's unit vector lies in the span of admitted sums. *Bisection*: a fresh session of threshold counts pins the integer balance.

Defense	Exact	Bisection
No audit	100 / 100	100 / 100
Minimum group, $k = 3$	100 / 100	100 / 100
Minimum group, $k = 5$	100 / 100	100 / 100
Minimum group, $k = 10$	100 / 100	100 / 100
Nested pairs, $k = 3$	100 / 100	0 / 100
Linear audit, $k = 3$	0 / 100	0 / 100

if the gateway refuses it. In the adaptive study a target is *disclosed* if the answers the gateway returned determine its value exactly, as an oracle applying Eq. 3 decides, whatever the attacker claims.

Statistical reporting. For a proportion we report the 95% Wilson score interval. Session-alarm thresholds are set per purpose on a held-out half of the benign sessions; detection rates are measured on the other half plus all attacks.

5.2 RQ1: Does a per-query minimum group size suffice?

It does not, on the workload we can regenerate. Snowflake and BigQuery were not queried: the repository has no credentials for them, so this paper does not report a vendor result.

Table 5 is a portfolio with no language model, 100 targets, at most 100 queries, on 1,500 synthetic customers. With no audit, a one-customer sum recovers every target. A minimum group size at $k = 3$, 5, and 10 refuses that sum and still loses every target to a segment complement, and a fresh session of threshold counts pins every balance. Nested pairs stop the complement and the bisection, and lose every target to three overlapping ranges. The linear audit recovers none (Wilson upper bound 3.7%). A sum of two customers, in a fresh session with no minimum group size, matched the two balances in 100 of 100 sessions and determined 0 of the customers. At $k = 3$ the linear audit refused that query every time, because the group has size 2. The finance sessions in Section 5.3 are the same comparison on the generated analyst workload: suppression refuses 0.6% of benign queries and stops 1.0% of two-query trackers; the linear audit refuses 3.3% and stops all of them.

5.3 RQ2: What does each test prevent, and at what cost?

The linear audit prevents every scripted differencing attack at the same benign cost as the weaker pairwise audit.

The audit refuses only the query that would complete an isolation (Eq. 5). Table 6 compares the four modes at $k = 3$ on the finance sessions, and Fig. 3 adds $k = 2$ and $k = 5$. The small-cell test alone stops almost no differencing. The pairwise audit stops two-query attacks but only 12.0% of three-query attacks, whose sets are not nested. The linear audit stops both and refuses 3.3% of benign queries, the same as the pairwise audit: honest workloads rarely produce exact linear dependencies that the small-cell and nested-pair tests do not already catch.

Table 6: Audit modes at $k = 3$ on the finance sessions. *Prevented*: attack sessions refused at or before the disclosing query, for two-query and three-query differencing. *Refused*: benign queries refused. *Sessions*: benign sessions with at least one refusal.

	Prevented		Benign cost	
Mode	Two-query	Three-query	Refused	Sessions
Stateless	1.0%	3.0%	0.6%	7.0%
Pairwise	100.0%	12.0%	3.3%	26.8%
Linear	100.0%	100.0%	3.3%	26.8%
Strict	100.0%	100.0%	25.6%	94.0%

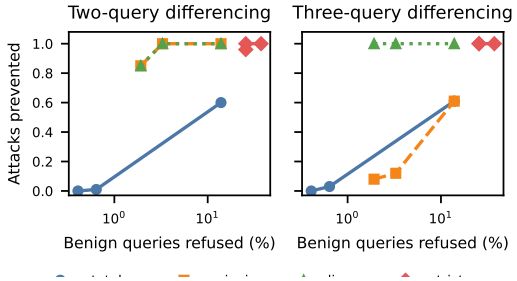


Figure 3: Query-level audit on finance sessions: share of attacks prevented vs. share of benign queries refused, for $k \in \{2, 3, 5\}$ and the four audit modes. Left: two-query differencing; right: three-query differencing whose sets are not nested. x axis symmetric-log.

Strict mode keeps the same protection but refuses 25.6% of benign queries, because analysts filter on balances (histogram bands, drill-downs above a threshold). At $k = 5$ every non-strict mode refuses 13.9% of benign queries, because ordinary nation-by-segment groupings contain fewer than five customers.

Filter test and join weights. The regression tests check both rules. A count of the form `COUNT(*) WHERE c_custkey = 570 AND c_acctbal > t` has a candidate set of one customer, so the filter test refuses the first probe. A `SUM` over `customer JOIN orders` is a weighted sum; the tests refuse the query that, together with the unweighted sum of the same customers, determines one balance, and they refuse a predicate `COUNT(*)` that counts those customers unequally. Query totals from an earlier unlogged probe are not repeated here.

5.4 RQ3: Does it hold against attackers that adapt or know its rules?

For exact disclosure it does on the cached attackers: under the linear audit no target was disclosed, and every weaker defense in those files was defeated. The rule-aware evidence in this repository is the portfolio in Section 5.2, not a separate combination-test study.

We ran 50 targets per defense with gpt-4o-mini and 25 with gpt-4o. Fig. 4 and Table 7 report disclosure as decided by the oracle. gpt-4o has no strict-mode file.

Every defense weaker than the linear audit was defeated. With the aggregate-only rule alone, both attackers succeed almost always,

Table 7: Cached adaptive attackers. *Disclosed*: share of targets whose exact balance the returned answers determine. *Refusals*: mean per episode. Each pair is gpt-4o-mini (50 targets) / gpt-4o (25 targets). gpt-4o has no strict-mode file.

Defense	Disclosed	Refusals
Aggregate-only rule	98.0% / 88.0%	0.2 / 0.0
Stateless audit	10.0% / 12.0%	2.2 / 2.9
Pairwise audit	24.0% / 24.0%	5.0 / 3.0
Linear audit	0.0% / 0.0%	5.1 / 3.8
Strict mode	0.0% / –	5.4 / –

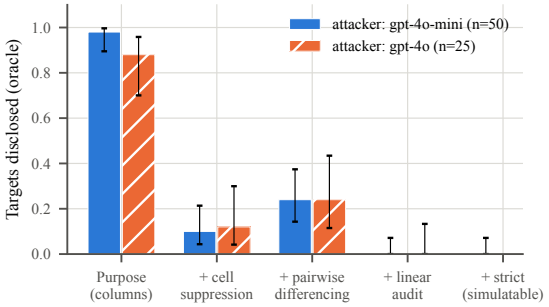


Figure 4: Adaptive LLM attackers: share of targets whose exact balance is determined by the answers the gateway returned (oracle, Eq. 3), per defense and attacker model. Bars: 95% Wilson intervals.

usually with one AVG over a single key. The stateless audit stops direct isolation but not differencing. The pairwise audit is defeated by combinations it does not model, such as overlapping key ranges. It was defeated more often than the stateless audit, though not significantly (Fisher's exact test, $p = 0.11$ and 0.46).

Under the linear audit no target was disclosed by either attacker: 0 of 50 and 0 of 25, with upper 95% bounds of 7.1% and 13.3%. Strict mode gives 0.0% disclosure for gpt-4o-mini, with refusals that by Proposition 4.4 carry no information about balances.

The cached files stop at the linear and strict rows above. A separate "final gateway" arm is not in those files, so it is not reported.

An attacker that knows the rules. The cached LLM attackers were not told the audit's algebra. The portfolio in Section 5.2 was: it uses the segment complement, the three-range overlap, and threshold bisection, and it is the measurement in this repository. A 200-target scripted study of two-customer sums under a combination test is not in the repository, so those counts are not reported. The two-customer sum that is reported determined neither customer when it was answered, and the linear audit at $k = 3$ refused it.

5.5 RQ4: Would alarms or noise do instead?

Session alarms do not replace the audit for differencing, and Laplace noise does not replace it when the answer must be exact. The two are complements. Session signals separate the attacks that show in the shape of a session, and they miss differencing, because honest sessions produce the same pattern. The sessions are length-matched, so a high score is not a long session.

Table 8: Session-alarm detection at a 1% benign false-positive rate, from the macros behind Section 5.5. Per query: the maximum of the per-query signals. Session: the accumulated score.

Family	Per query	Session
Drift	47.0%	66.0%
Enumeration	87.0%	100.0%
Two-query differencing	0.0%	0.0%
Three-query differencing	0.0%	0.0%

We computed the AUROC of every signal for every attack family against held-out benign sessions of the same purpose. Session length carries no label information (query-count AUROC 0.53–0.61). Three families are separated almost perfectly by the signal that measures their mechanism: drift by use of columns outside the purpose, enumeration by breadth, and small cells by the per-query cell test.

At a 1% benign false-positive rate, session state helps where the violation is cumulative: drift detection rises from 47.0% to 66.0%, and enumeration detection from 87.0% to 100.0%.

For the aggregate families, session alarms detect almost nothing at this rate (Table 8), because 13.8% of benign finance sessions (55 of 400) genuinely isolate one customer, typically with an average by nation and segment followed by the same average restricted to positive balances, the pattern of Fig. 1a. The right tool refuses the single query that completes the isolation, as the audit does (Section 5.3).

Noise instead of refusal. On groups of about a dozen customers, none of the budgets we tried with the Laplace mechanism gave both useful answers and protection, while the audit returns exact answers and refuses a few queries.

Instead of refusing, a gateway could answer every aggregate with Laplace noise [16, 24]. We clip balances to $[-1{,}000, 10{,}000]$ (sensitivity 10,000 per SUM), split a session budget ε equally across its protected aggregates, and answer an average as a noisy sum over the exact group size. Table 9 compares utility on the benign finance sessions with the share of differencing sessions in which the attacker's estimate lands within 10% of the target's balance.

In this workload no budget we tried gives both. At $\varepsilon = 10$ only 37.0% of benign answers are within 5% of the true value. At $\varepsilon = 100$ most are, but the attacker then succeeds in 18.7% of sessions. The audit instead refuses 3.3% of benign queries and returns the rest exactly.

The comparison is narrow: it covers one mechanism, one clipping range and small groups, and does not show that differential privacy cannot serve such workloads, nor that the audit gives equivalent protection. Differential privacy bounds inference for every query and needs no entity sets, while the audit's guarantee covers exact disclosure through sums only (Table 3). They are complementary. A Laplace mechanism that also suppresses a group unless a noisy count clears $k = 3$, with basic composition and not an OpenDP or DPSQL+ accountant, is in the appendix: at $\varepsilon = 100$ the released answers have median relative error 1.8% and the two-query difference is within 10% of the balance on 11.2% of draws. No budget there returns the exact answers the audit returns.

Table 9: Differential privacy vs. the session audit on the finance workload. *Median error, Within 5%*: relative error of benign answers per output group. *Refused*: benign queries refused. *Attacker*: share of differencing sessions whose estimate is within 10% of the target's balance (audit: midpoint of the linear-programming bound; guessing the domain midpoint scores 11.5%).

Answering	Median error	Within 5%	Refused	Attacker
DP, $\varepsilon = 1$	87.7%	8.1%	–	0.1%
DP, $\varepsilon = 10$	8.8%	37.0%	–	2.2%
DP, $\varepsilon = 100$	0.9%	91.6%	–	18.7%
Linear audit, $k = 3$	0%	100%	3.3%	11.5%

5.6 RQ5: What does it cost on human-written queries and in deployment?

On BIRD's gold queries the audit costs only its per-query rules: the stateful tests refused nothing more. An agent re-asking those questions, and Spider 2.0, are not in this repository, so this paper does not report them.

Fig. 5 and Table 10 use the committed BIRD log: 106 `financial` queries and 64 `debit_card_specializing` queries, one audit version.

The linear audit refuses 19.6% of queries on `financial` and 41.5% on `debit_card_specializing`, and in both databases the stateless audit refuses the same share. The higher rate on the debit-card database reflects its questions, many of which ask for one customer's consumption; under an aggregate-only purpose those are disclosures by design, so the rate measures policy fit, not audit error.

To see why queries are refused, we ran each one alone in a fresh session (lower part of Table 10). Most refusals come from the filter test. To judge whether the remaining refusals are errors, we classified each of the 40 queries refused alone from its question text. 30 ask for one entity's value, for the top or bottom entity, or look up an exact amount to identify someone; under an aggregate-only purpose these are disclosures by design. 7 are false positives, mostly counts over a join that the equal-weight rule refuses, and 3 are borderline, such as revenue per gas station where some stations have few customers. The combination test leaves both refusal rates unchanged.

Deployment over MCP. We loaded the TPC-H-shaped schema into PostgreSQL 16 at scale factor 1 (150,000 customers), generated 260 sessions (2,942 queries), and replayed them through the gateway and through a second MCP hop. Every attack family was refused at or before its disclosing query, and every decision matched the library. Table 11 is that log. On the direct path 112 of 2942 checks took over one second and the slowest took 4.44 s. A one-million-customer replay is not in the repository. A separate DuckDB timing, not that missing run, aggregated key weights for one nation of a million-customer table in 9.4 ms, and 1000 short range updates peaked at 3.71 ms (appendix). Medians of check, execute, and transport do not add to the median round trip: on the queries that executed, the three medians sum to 14.0 ms and the median round trip is 53.7 ms.

Table 10: Human-written BIRD workloads under the analytics purpose, $k = 3$. Upper: sessions of 6–16 queries. Lower: each query alone in a fresh session. Per-reason counts are in the appendix.

	`financial`	`debit_card`
Queries	106	64
Refused: raw protected value	3.8%	3.1%
Refused: stateless audit	19.6%	41.5%
Refused: linear audit	19.6%	41.5%
with conditional split	16.6%	37.3%
Refused: strict mode	21.8%	41.5%
Entity sets computed	81.8%	90.3%
Check time, ms (median / 95th)	0.72 / 56.2	0.69 / 155.6
Query time, ms (median / 95th)	0.37 / 138.4	0.51 / 65.8
Queries refused alone	19	27
without the filter test	6	21
with conditional split	16	24

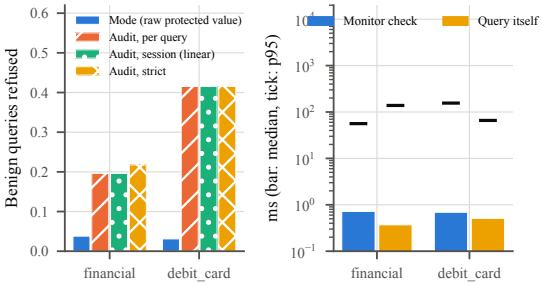


Figure 5: Human-written BIRD workloads (200 sessions per database). Left: share of benign queries refused. Right: monitor check time vs. query execution time (bars: median; ticks: 95th percentile; log scale).

Table 11: End-to-end MCP deployment on PostgreSQL, TPC-H scale factor 1 (260 sessions, 2,942 queries). *Parity*: decisions equal to the in-process library. Times in ms, median / 95th percentile.

Path	Parity	Check	Execute	Transport	Round trip
Gateway → Postgres	100.0%	1.4 / 741	11.5 / 182	1.0	26.7 / 758
Gateway → DB MCP → Postgres	100.0%	1.8 / 923	12.2 / 179	0.9	27.6 / 937

6 DISCUSSION

6.1 Deployment

As an MCP proxy the gateway leaves agents and databases unchanged. A deployment chooses the account, the protected columns, k, strict mode and log retention, and should not describe the audit's rules to the agent (Section 5.6).

6.2 Limitations

What the audit does not guarantee. Propositions 4.1 and 4.2 rule out exact linear disclosure of fewer than k entities. The finance-session modes were measured at $k \in \{2, 3, 5\}$. Intervals are not bounded [30] (none narrowed in Table 9). Filters can reveal the

multiset of a group of size at least k; only strict mode removes that. Table 3 lists the rest.

Validity. The adaptive study has 50 and 25 targets (upper bounds 7.1% and 13.3%), a budget of 15, and no bisection; `claude-sonnet-5-5` sent no attack SQL. We authored the TPC-H sessions and the 100-target portfolio. BIRD here is gold SQL, not an agent re-asking the questions. Snowflake, BigQuery, Spider 2.0, and a one-million-customer PostgreSQL replay are not in the repository. The 10^{-9} test misses a constructed case near weights of 10^9. Strict mode refuses 25.6% of benign finance queries.

7 CONCLUSION

The session audit refuses the query that would make one person's protected value exact. On the finance sessions that refuses 3.3% of benign queries, against 0.6% for a per-query minimum group size, and a 100-target portfolio that defeats the minimum group size recovers nothing from the linear audit. A sum of two customers, when answered, is a sum. The bound is exact recovery, per identity, on a static snapshot. Vendor warehouses and Spider 2.0 were not measured in this repository.

REFERENCES

[1] Nabil R. Adam and John C. Worthmann. 1989. Security-control methods for statistical databases: a comparative study. *Comput. Surveys* 21, 4 (1989), 515–556. https://doi.org/10.1145/76894.76895

[2] Rakesh Agrawal, Roberto Bayardo, Christos Faloutsos, Jerry Kiernan, Ralf Rantzau, and Ramakrishnan Srikant. 2004. Auditing Compliance with a Hippocratic Database. In *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. 516–527.

[3] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. 2002. Hippocratic Databases. In *VLDB '02: Proceedings of the 28th International Conference on Very Large Databases*. Elsevier, 143–154. https://doi.org/10.1016/b978-155860869-6/50021-4

[4] Amazon Web Services. [n.d.]. AWS Clean Rooms Differential Privacy. https://docs.aws.amazon.com/clean-rooms/latest/userguide/differential-privacy.html. AWS Clean Rooms User Guide, accessed November 2026.

[5] Amazon Web Services. [n.d.]. Minimum aggregation thresholds. https://docs.aws.amazon.com/clean-rooms/latest/userguide/custom-min-agg-thresholds.html. AWS Clean Rooms User Guide, accessed November 2026.

[6] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2001. Why and Where: A Characterization of Data Provenance. In *Database Theory — ICDT 2001*. 316–330. https://doi.org/10.1007/3-540-44503-X_20

[7] Ji-Won Byun and Ninghui Li. 2006. Purpose based access control for privacy protection in relational database systems. *The VLDB Journal* 17, 4 (2006), 603–619. https://doi.org/10.1007/s00778-006-0023-0

[8] F.Y. Chin and G. Ozsoyoglu. 1982. Auditing and Inference Control in Statistical Databases. *IEEE Transactions on Software Engineering* SE-8, 6 (1982), 574–582. https://doi.org/10.1109/tse.1982.236161

[9] Aloni Cohen and Kobbi Nissim. 2020. Linear Program Reconstruction in Practice. *Journal of Privacy and Confidentiality* 10, 1 (2020). https://doi.org/10.29012/jpc.711

[10] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. *arXiv preprint arXiv:2505.23643* (2025).

[11] Ana-Maria Cretu, Florimond Houssiau, Antoine Cully, and Yves-Alexandre de Montjoye. 2022. QuerySnout: Automating the Discovery of Attribute Inference Attacks against Query-Based Systems. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 623–637. https://doi.org/10.1145/3548606.3560581

[12] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating Prompt Injections by Design. In *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 587–618. https://doi.org/10.1109/satml68715.2026.00040

[13] Dorothy E. Denning and Peter J. Denning. 1979. The tracker: a threat to statistical database security. *ACM Transactions on Database Systems* 4, 1 (1979), 76–96. https://doi.org/10.1145/320064.320069

[14] Irit Dinur and Kobbi Nissim. 2003. Revealing information while preserving privacy. In *Proceedings of the twenty-second ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. ACM, 202–210. https://doi.org/10.1145/773153.773173

[15] David Dobkin, Anita K. Jones, and Richard J. Lipton. 1979. Secure databases: protection against user influence. *ACM Transactions on Database Systems* 4, 1 (1979), 97–106. https://doi.org/10.1145/320064.320068

[16] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *Lecture Notes in Computer Science*. Springer Berlin Heidelberg, 265–284. https://doi.org/10.1007/11681878_14

[17] Yang Fei, Yangfan Jiang, Yin Yang, and Xiaokui Xiao. 2026. Benchmarking Text-to-SQL under Role-Based Access Control. *arXiv preprint arXiv:2607.22115* (2026).

[18] Paul Francis, Sebastian Probst Eide, and Reinhard Munz. 2017. Diffix: High-Utility Database Anonymization. In *Lecture Notes in Computer Science*. Springer International Publishing, 141–158. https://doi.org/10.1007/978-3-319-67280-9_8

[19] Andrea Gadotti, Florimond Houssiau, Luc Rocher, Benjamin Livshits, and Yves-Alexandre de Montjoye. 2018. When the signal is in the noise: Exploiting Diffix's Sticky Noise. *arXiv preprint arXiv:1804.06752* (2018). In Proc. USENIX Security 2019.

[20] Google Cloud. [n.d.]. Restrict data access using analysis rules. https://docs.cloud.google.com/bigquery/docs/analysis-rules. BigQuery documentation, accessed November 2026.

[21] Google Cloud. [n.d.]. Share sensitive data with data clean rooms. https://docs.cloud.google.com/bigquery/docs/data-clean-rooms. BigQuery documentation, accessed November 2026.

[22] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. 2007. Provenance Semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. 31–40. https://doi.org/10.1145/1265530.1265535

[23] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. ACM, 79–90. https://doi.org/10.1145/3605764.3623985

[24] Noah Johnson, Joseph P. Near, and Dawn Song. 2018. Towards practical differential privacy for SQL queries. *Proceedings of the VLDB Endowment* 11, 5 (2018), 526–539. https://doi.org/10.1145/3187009.3177733

[25] Ashish Kamra, Evimaria Terzi, and Elisa Bertino. 2007. Detecting anomalous access patterns in relational databases. *The VLDB Journal* 17, 5 (2007), 1063–1077. https://doi.org/10.1007/s00778-007-0051-4

[26] Raghav Kaushik and Ravishankar Ramamurthy. 2011. Efficient Auditing for Complex SQL Queries. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 697–708. https://doi.org/10.1145/1989323.1989396

[27] Krishnaram Kenthapadi, Nina Mishra, and Kobbi Nissim. 2005. Simulatable auditing. In *Proceedings of the twenty-fourth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. ACM, 118–127. https://doi.org/10.1145/1065167.1065183

[28] Jon Kleinberg, Christos Papadimitriou, and Prabhakar Raghavan. 2003. Auditing Boolean attributes. *J. Comput. System Sci.* 66, 1 (2003), 244–253. https://doi.org/10.1016/s0022-0000(02)00036-3

[29] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. *arXiv preprint arXiv:2305.03111* (2023). NeurIPS 2023 Datasets and Benchmarks Track.

[30] Yingjiu Li, Lingyu Wang, X. Sean Wang, and Sushil Jajodia. 2002. Auditing Interval-Based Inference. In *Advanced Information Systems Engineering (CAiSE) (Lecture Notes in Computer Science)*, Vol. 2348. 553–568. https://doi.org/10.1007/3-540-47961-9_38

[31] Ruiheng Liu, XiaoBing Chen, Jinyu Zhang, Qiongwen Zhang, Yu Zhang, and Bailong Yang. 2026. SafeNLIDB: A Privacy-Preserving Safety Alignment Framework for LLM-based Natural Language Database Interfaces. *arXiv preprint arXiv:2511.06778* (2026). AAAI 2026 extended version.

[32] Tomoya Matsumoto, Shokichi Takakura, Shun Takagi, and Satoshi Hasegawa. 2026. DPSQL+: A Differentially Private SQL Library with a Minimum Frequency Rule. *arXiv preprint arXiv:2602.22699* (2026). Accepted to TPDP 2026.

[33] Microsoft. [n.d.]. Dynamic Data Masking. https://learn.microsoft.com/en-us/sql/relational-databases/security/dynamic-data-masking. SQL Server documentation, accessed November 2026.

[34] Rajeev Motwani, Shubha U. Nabar, and Dilys Thomas. 2008. Auditing SQL Queries. In *Proceedings of the 24th International Conference on Data Engineering (ICDE)*. 287–296. https://doi.org/10.1109/ICDE.2008.4497437

[35] Shubha U. Nabar, Krishnaram Kenthapadi, Nina Mishra, and Rajeev Motwani. 2008. A Survey of Query Auditing Techniques for Data Privacy. In *Privacy-Preserving Data Mining: Models and Algorithms*. Springer, 415–431. https://doi.org/10.1007/978-0-387-70992-5_17

[36] Shubha U. Nabar, Bhaskara Marthi, Krishnaram Kenthapadi, Nina Mishra, and Rajeev Motwani. 2006. Towards Robustness in Query Auditing. In *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB)*. 151–162.

[37] Oracle. [n.d.]. Why Choose Oracle Deep Data Security. https://docs.oracle.com/en/database/oracle/oracle-database/26/ddscg/why-choose-oracle-deep-data-security.html. Oracle AI Database 26ai documentation, accessed November 2026.

[38] Rodrigo Pedro, Daniel Castro, Paulo Carreira, and Nuno Santos. 2023. From Prompt Injections to SQL Injection Attacks: How Protected is Your LLM-Integrated Web Application? *arXiv preprint arXiv:2308.01990* (2023).

[39] Shariq Rizvi, Alberto Mendelzon, S. Sudarshan, and Prasan Roy. 2004. Extending query rewriting techniques for fine-grained access control. In *Proceedings of the 2004 ACM SIGMOD international conference on Management of data*. ACM, 551–562. https://doi.org/10.1145/1007568.1007631

[40] Mark Russinovich, Ahmed Salem, and Ronen Eldan. 2024. Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack. *arXiv preprint arXiv:2404.01833* (2024). In Proc. USENIX Security 2025.

[41] Mushtari Sadia, Ang Chen, and Amrita Roy Chowdhury. 2026. PIMENTO: A Privacy Framework for Querying Text. *arXiv preprint arXiv:2609.14745* (2026).

[42] R.S. Sandhu, E.J. Coyne, H.L. Feinstein, and C.E. Youman. 1996. Role-based access control models. *Computer* 29, 2 (1996), 38–47. https://doi.org/10.1109/2.485845

[43] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. *arXiv preprint arXiv:2504.11703* (2025).

[44] Basavesh Ammanaghatta Shivakumar, Swarn Priya, and Peng Gao. 2026. Agenra: A Language and Verified Core for Stateful LLM Agent Authorization. *arXiv preprint arXiv:2608.22868* (2026).

[45] Snowflake Inc. [n.d.]. Aggregation policies. https://docs.snowflake.com/en/user-guide/aggregation-policies. Snowflake documentation, accessed November 2026.

[46] Snowflake Inc. [n.d.]. Differential privacy in Snowflake Data Clean Rooms. https://docs.snowflake.com/en/user-guide/cleanrooms/differential-privacy. Snowflake documentation, accessed November 2026.

[47] Yanqi Song, Ruiheng Liu, Shu Chen, Qianhao Ren, Yu Zhang, and Yongqi Yu. 2024. SecureSQL: Evaluating Data Leakage of Large Language Models as Natural Language Interfaces to Databases. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 5975–5990. https://doi.org/10.18653/v1/2024.findings-emnlp.346

[48] Bozhidar Stevanoski, Ana-Maria Cretu, and Yves-Alexandre de Montjoye. 2024. QueryCheetah: Fast Automated Discovery of Attribute Inference Attacks against Query-Based Systems. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 3451–3465. https://doi.org/10.1145/3658644.3690272

[49] Rakesh Kumar Surapani, Pradeep Kumar Dolabehera Kakitapelli, Arun Morampudi, and Praveena Padi. 2026. Authorization Architectures for Tool-Using AI Agents. *arXiv preprint arXiv:2609.15906* (2026).

[50] Latanya Sweeney. 2002. k-anonymity: a model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* 10, 05 (2002), 557–570. https://doi.org/10.1142/s0218488502001648

[51] Andreas M. Tillmann and Marc E. Pfetsch. 2014. The Computational Complexity of the Restricted Isometry Property, the Nullspace Property, and Related Concepts in Compressed Sensing. *IEEE Transactions on Information Theory* 60, 2 (2014), 1248–1259. https://doi.org/10.1109/TIT.2013.2290112

[52] Kuncan Wang, Ziting Wang, Peizhuo Lv, Haoyang Li, Guoliang Li, Gao Cong, and Wei Dong. 2026. Data Agents Under Attack: Vulnerabilities in LLM-Driven Analytical Systems. *arXiv preprint arXiv:2606.08661* (2026).

[53] Minmin Wu. 2026. FactGate: A Task-Scoped Data Gateway for AI Agents with Cumulative Exposure Accounting. *arXiv preprint arXiv:2607.00751* (2026). Version 2. Version 1 was titled SessionBound.

[54] Chen Yueh-Han, Nitish Joshi, Yulin Chen, Maksym Andriushchenko, Rico Angell, and He He. 2025. Monitoring Decomposition Attacks in LLMs with Lightweight Sequential Monitors. *arXiv preprint arXiv:2506.10949* (2025). In Proc. ICLR 2026.

[55] Đorđe Klisura and Anthony Rios. 2024. Unmasking Database Vulnerabilities: Zero-Knowledge Schema Inference Attacks in Text-to-SQL Systems. *arXiv preprint arXiv:2406.14545* (2024). In Findings of the Association for Computational Linguistics: NAACL 2025.